# SLAP
# Simple Linear Algebra Package

Andrea Marchi

15 marzo 2026

## Indice

**1 Data type** **2**

**2 Basic operations** **4**
2.1 Equality . . . . . 4
2.2 Addition and sottration . . . . . 4
2.3 Multiplication . . . . . 4
2.3.1 Scalar-matrix multiplication . . . . . 4
2.3.2 Vector-matrix multiplication . . . . . 4
2.3.3 Matrix-matrix multiplication . . . . . 5

**3 Functions** **10**
3.1 Matrix norm . . . . . 10

**4 Gauss elimination** **11**
4.1 Basic operations . . . . . 12
4.1.1 Row swapping . . . . . 12
4.1.2 Scalar multiplication of rows . . . . . 12
4.1.3 Row addition . . . . . 13

**5 Matrix factorizations** **13**
5.1 LU(P) decomposition . . . . . 14
5.2 QR decomposition . . . . . 15
5.3 Cholesky decomposition . . . . . 17
5.4 Singular value decomposition (SVD) . . . . . 17
5.5 Spectral decomposition . . . . . 17

5.6 Schur decomposition . . . . . 17
5.7 Principal component analysis (PCA) . . . . . 17
5.8 Non-negative factorization . . . . . 17

**6 Solve linear systems** **17**
6.1 Gauss-Jordan elimination . . . . . 17
6.1.1 Backward substitution . . . . . 17
6.1.2 Forward substitution . . . . . 18
6.2 LU(P) decomposition . . . . . 19
6.3 QR decomposition . . . . . 20
6.4 Conjugate gradient method . . . . . 22

**7 Determinante** **26**
7.1 Formula di Leibniz . . . . . 26
7.2 Sviluppo di Laplace . . . . . 27
7.3 Algoritmo di Gauss . . . . . 27
7.4 Determinante tramite LU . . . . . 27

**8 Inverse matrix** **28**
8.1 Tramite soluzione del sistema . . . . . 28

**9 Eigen** **30**
9.1 QR algorithm . . . . . 30
9.2 Metodo delle potenze . . . . . 32

**A Random numbers** **36**
A.1 Sampling from Gaussian distribution . . . . . 36

**B Stress test** **36**
B.1 Matrici di Hilbert . . . . . 36
B.2 Matrice quasi instabile . . . . . 36

# 1 Data type

Le matrici sono salvate in array monodimensionali *row-major*, ovvero che i dati sulle righe sono sequenziali. Quindi nella indicizzazione degli elementi della matrice con $n$ righe e $m$ colonne ($\mathbb{R}^{n \times m}$) si usa la formula:

```
matrix[i][j] = array[i*m + j]
```

Le righe vanno da 0 a $n-1$, mentre le colonne vanno da 0 a $m-1$.

Alcune librerie usano la notazione *column-major*, ovvero con indicizzazione `array[j*n+i]`. Avere una funzione che organizza i dati della matrice in colonne o in righe è comodo per quanto riguarda l'ottimizzazione della cache di lettura dei dati sequenziali dalla RAM alla CPU.

Il tipo di dati (essendo in C) è una struttura (`struct`) e la definizione cambia al variare del tipo di dati base (il C non permette l'uso di template). La struttura base è:

```
typedef struct _matd{
        unsigned int n_rows;
        unsigned int n_cols;
        double *data; // row-major matrix data array
} matd;
```

dove la lettera finale indica il tipo di variabile usata, in questo caso `double`. I tipi di dati che ha senso utilizzare nella libreria sono:

**d** virgola mobile a doppia precisione (`double`)
**f** virgola mobile (`float`)
**i** intero (`int`)
**b** byte (`unsigned char`)

Per allocare la memoria e liberarla (sempre liberare la memoria dopo averla allocata):

```
matd* new_matd(unsigned int num_rows, unsigned int num_cols)
{
        int i;
        // create a new double matrix
        if(num_rows == 0) { /*SLAP_ERROR(INVALID_ROWS);*/ return 0; }
             // dovrebbe ritornare NULL
        if(num_cols == 0) { /*SLAP_ERROR(INVALID_COLS);*/ return 0; }
             // dovrebbe ritornare NULL

        matd *m = calloc(1, sizeof(*m)); // allocate space for the
            struct
        // CONTROLLARE LA MATRICE CREATA ( SLAP_CHECK(m) )
        m->n_rows = num_rows;
        m->n_cols = num_cols;
        m->data = calloc(m->n_rows*m->n_cols, sizeof(*m->data));
        // CONTROLLARE I DATI CREATI ( SLAP_CHECK(m->data) )
        for(i=0; i<num_rows*num_cols; i++) m->data[i] = 0; // set to
            zero

        return m;
}
```

```

void free_mat(matd *matrix)
{
        free(matrix->data); // delete the data
        free(matrix); // delete the data structure
}
```

Come setters e getters non potendo usare le operation del C++ e non riuscendo a fare qualcosa di funzionante e decente con le macro[1] uso le funzioni

```
double matd_get(matd matrix, unsigned int row, unsigned int col) {
        return matrix.data[row*matrix.n_cols + col]; // row-major
}
void matd_set(matd matrix, unsigned int row, unsigned int col, double
     val) {
        matrix.data[row*matrix.n_cols + col] = val; // row-major
}
```

Dovrei controllare che l'accesso sia corretto (che non cerchi di scrivere/leggere dati non allocati).

# 2 Basic operations

Mettere un pannello (anche una tabella) che riassume le operazioni, il nome delle funzioni e come si usano.

## 2.1 Equality

## 2.2 Addition and sottration

## 2.3 Multiplication

### 2.3.1 Scalar-matrix multiplication

### 2.3.2 Vector-matrix multiplication

La moltiplicazione matrice-vettore può essere vista come la moltiplicazione tra una matrice e un'altra nella forma di un vettore colonna. Fare un controllo numerico su questa affermazione

[1] Usare le macro mi permetterebbe di risparmiare tempo nella allocazione dei parametri delle funzioni. Negli algoritmi potrei usare l'accesso diretto all'array `data`.

### 2.3.3 Matrix-matrix multiplication

Il prodotto tra due matrici $\mathbf{A} \in \mathbb{R}^{n \times m}$ (matrice con $n$ righe e $m$ colonne) $\mathbf{B} \in \mathbb{R}^{m \times p}$ ($m$ righe, $p$ colonne) è una matrice $\mathbf{C} \in \mathbb{R}^{n \times p}$ le cui componenti sono definite come:

$$C_{ij} = \sum_{k=1}^{m} A_{ik} \, B_{kj} \tag{1}$$

Applicando direttamente la definizione della moltiplicazione tra matrici si ottiene un algoritmo che ha un'efficienza computazionale di $O(n^3)$ (dove $n$ è la dimensione di una matrice quadrata $n \times n$). Migliori limiti asintotici sono stati scoperti dopo l'algoritmo di Strassen negli anni '60 (il limite teorico rimane ancora ignoto). Recentemente è stato annunciato un algoritmo con l'efficienza teorica di $O(n^{2.37188})$, ma si tratta di un *algoritmo galattico*, ovvero che richiede una dimensione talmente grande delle matrici per funzionare con tale efficienza da non essere praticamente realizzabile (in questo caso a causa di una grossa costante).

**Simple algorithm:** Dalla definizione della moltiplicazione tra matrici si ricava direttamente l'algoritmo 4.

**Algorithm 1** Semplice algoritmo per la moltiplicazione tra matrici

```
Require: matrix A, matrix B
C ← matrix of the appripriate size
for i ← 1 to n do          ▹ nell'implementazione parte da 0 e arriva a n − 1
    for j ← 1 to p do                ▹ stessa considerazione dell'indice i
        sum ← 0
        for k ← 1 to m do
            sum ← sum + A_ik B_kj
        end for
        C_ij ← sum
    end for
end for
```

Questo algoritmo richiede un tempo pari a $O(nmp)$ (se le matrici sono quadrate $n \times n$ il tempo diventa $O(n^3)$).

**Cache optimization:** I tre cicli all'interno della moltiplicazione possono essere scambiati senza modificare il risultato in termini matematici, tuttavia questo può avere un considerevole impatto in termini di implementazione

ed in particolare in termini di tempo di esecuzione su elaboratori elettronici. Infatti i computer hanno degli algoritmi ottimizzati per l'accesso dei dati dalla RAM alla CPU quando questi sono sequenziali (uno di fila all'altro in RAM). Quando il dato successivo necessario all'algoritmo non è quello immediatamente successivo in RAM si parla di *cache miss*. La versione ottimale dell'algoritmo semplice per due matrici **A** e **B** in row-major è la versione tiled, dove le matrici sono implicitamente divise in *tiles* di dimensione $\sqrt{N} \times \sqrt{N}$, è descritta nell'algoritmo 2.

**Algorithm 2** Semplice algoritmo per la moltiplicazione tra matrici con gestione migliorata della cache non mi torna un granche'

**Require:** matrix **A**, matrix **B**

```
C ← matrix of the appripriate size
tile size T ← √N
for I ← 1 to n in steps of T do
    for J ← 1 to p in steps of T do
        for K ← 1 to m in steps of T do
            for i ← I to min(I + T, n) do          ▷ Multiply A_{I:I+T,K:K+T} and B_{K:K+T,J:J+T} into C_{I:I+T,J:J+T}
                for j ← J to min(J + T, p) do
                    sum ← 0
                    for k ← K to min(K + T, m) do
                        sum ← sum + A_ik B_kj
                    end for
                    C_ij ← C_ij + sum
                end for
            end for
        end for
    end for
end for
```

Nel modello di cache ideale l'algoritmo incorre in solamente $O(\frac{n^3}{b\sqrt{N}}$. Per le macchine moderne il denominatore $m\sqrt{N}$ ammonta a diversi ordini di grandezza, dunque il tempo necessario è quello effettivo del calcolo, invece di perdere tempo in *cache misses*.

. . . . . .

In `SLAP.h` abbiamo definito le matrici come *Row-Major* (memorizzate per righe). Significa che gli elementi di una riga sono adiacenti in memoria.

Nel tuo ciclo di moltiplicazione originale (`r,c,i`), il ciclo più interno è `i`. Mentre cerchi di calcolare `m2->data[i*m2->n_cols+c]`, la variabile i aumenta, facendoti "saltare" da una riga all'altra di `m2`. In MS-DOS (ma anche sui PC moderni!), saltare in continuazione tra indirizzi di memoria distanti costringe il processore a svuotare e ricaricare continuamente la cache (o a leggere a frammenti lenti dalla RAM).

La Soluzione: Algoritmo "Cache-Friendly" (`i-k-j` o `r-i-c`). Possiamo velocizzare drasticamente la funzione scambiando l'ordine dei cicli. Se mettiamo `c` come ciclo più interno, la memoria verrà letta e scritta in modo strettamente sequenziale per entrambe le matrici. Ecco il codice ottimizzato:

```
mat* mat_mul(const mat* m1, const mat* m2)
{
        mat *m;
        int r, c, i;
        TYPE m1_val;

        // Controllo compatibilita' dimensioni
        if(!(m1->n_cols == m2->n_rows)){
//              SLAP_ERROR(CANNOT_MULTIPLY);
                return NULL;
        }

        // mat_new inizializza gia' tutti i valori a 0, che e' esattamente
        // cio' che ci serve per accumulare le somme parziali.
        m = mat_new(m1->n_rows, m2->n_cols);
        if(!m) return NULL; // Controllo sicurezza allocazione

        // Cicli ottimizzati per Cache (Row-Major): R -> I -> C
        for(r = 0; r < m1->n_rows; r++) {
                for(i = 0; i < m1->n_cols; i++) {
                        // Questo valore rimane costante per tutto l'ultimo ciclo
                        m1_val = m1->data[r * m1->n_cols + i];

                        // OTTIMIZZAZIONE EXTRA: se m1_val e' "zero", possiamo saltare l'intera riga!
                        // Questo accelera incredibilmente il calcolo con matrici sparse o triangolari.
                        if (fabs(m1_val) < SLAP_MIN_COEF) continue;

                        for(c = 0; c < m2->n_cols; c++) {
                                // Ora SIA m->data SIA m2->data vengono percorsi in modo 100% sequenziale (c++).
                                // Il processore puo' pre-caricare la memoria e volare sui calcoli.
```

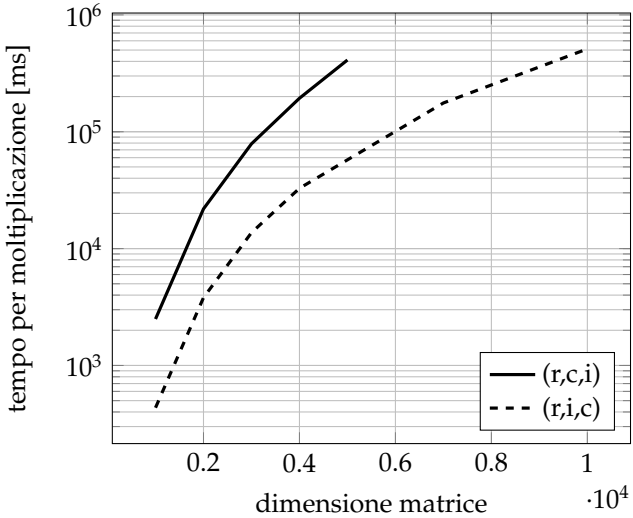
```
                                    m->data[r * m->n_cols + c] += m1_val
                                        * m2->data[i * m2->n_cols + c];
                            }
                    }
            }

            return m;
}
```

Cosa abbiamo migliorato:

- Zero Cache-Misses nel core loop: Nel ciclo `c`, `r*m->n_cols` e `i*m2->n_cols` sono costanti pre-calcolate. La CPU deve solo fare `array[c] += costante * array2[c]`. È l'operazione più amichevole che esista per un processore.
- Lo "Zero-Skip": Ho aggiunto `if (fabs(m1_val) < SLAP_MIN_COEF) continue;`. Poiché l'algebra lineare abbonda di matrici con molti zeri (matrici identità, matrici triangolari di Gauss, ecc.), se il moltiplicatore a sinistra è `0`, non ha senso calcolare il prodotto per le 100 o 1000 colonne successive. Questa riga singola fa risparmiare migliaia di operazioni.

Potrebbe essere fatto anche con lo stesso codice di prima, ma considerando la trasposta della matrice $B$.

Ecco un grafico di confronto tra la soluzione (r,c,i) e la soluzione (r,i,c)




I risultati sono trovati con MARK2-PC.

Praticamente si riduce il tempo di calcolo di un ordine di grandezza.

**Divide-and-conquer algorithm:** Una alternativa è l'algoritmo *dividi e conquista* per la moltiplicazione delle matrici che fa affidamento nella partizione delle matrici in blocchi

$$\begin{bmatrix} \mathbf{C}_{11} & \mathbf{C}_{12} \\ \mathbf{C}_{12} & \mathbf{C}_{22} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{12} & \mathbf{A}_{22} \end{bmatrix} \begin{bmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{12} & \mathbf{B}_{22} \end{bmatrix} = \\ = \begin{bmatrix} \mathbf{A}_{11}\,\mathbf{B}_{11} + \mathbf{A}_{12}\,\mathbf{B}_{21} & \mathbf{A}_{11}\,\mathbf{B}_{12} + \mathbf{A}_{12}\,\mathbf{B}_{22} \\ \mathbf{A}_{21}\,\mathbf{B}_{11} + \mathbf{A}_{22}\,\mathbf{B}_{21} & \mathbf{A}_{21}\,\mathbf{B}_{12} + \mathbf{A}_{22}\,\mathbf{B}_{22} \end{bmatrix} \tag{2}$$

che si applica a tutte le matrici quadrate che hanno dimensione pari a una potenza di due ($2^n \times 2^n$). L'algoritmo dividi e conquista calcola le moltiplicazioni più piccole ricorsivamente come un prodotto scalare $C_{11} = A_{11}B_{11}$ come caso base.

Una variante che funziona per matrici non-quadrate si basa sul dividere le matrici in due invece che in quattro. In questo caso si tratta di dividere le matrici in due parti uguali, o comunque il più vicino possibile a due parti uguali nel caso di dimensioni dispari.

**Algorithm 3** Moltiplicazione tra metrici tramite *divide-and-conquer*

**Require:** matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$, matrix $\mathbf{B} \in \mathbb{R}^{m \times p}$

if $\max(n, m, p)$ is below some threshold, use an unrolled version of the simple algorithm
**if** $\max(n, m, p) = n$ **then**
    $\mathbf{C} = \begin{bmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \end{bmatrix} \mathbf{B} = \begin{bmatrix} \mathbf{A}_1\mathbf{B} \\ \mathbf{A}_2\mathbf{B} \end{bmatrix}$ ▹ split **A** horizontally
**else if** $\max(n, m, p) = p$
    $\mathbf{C} = \mathbf{A} \begin{bmatrix} \mathbf{B}_1 & \mathbf{B}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{A}\mathbf{B}_1 \\ \mathbf{A}\mathbf{B}_2 \end{bmatrix}$ ▹ split **B** vertically
**else** ▹ ($\max(n, m, p) = m$)
    $\mathbf{C} = \begin{bmatrix} \mathbf{A}_1 & \mathbf{A}_2 \end{bmatrix} \begin{bmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{bmatrix} = \mathbf{A}_1\mathbf{B}_1 + \mathbf{A}_2\mathbf{B}_2$ ▹ split **A** vertically and **B** horizontally
**end if**

**Strassen algorithm** L'osservazione chiave è che per matrici $2 \times 2$ le moltiplicazioni necessarie sono 7 invece che 8, al costo di 11 operazioni aggiuntive tra somme e sottrazioni. Quindi si tratta di un algoritmo *dividi e conquista* in cui la matrici di input sono suddivise in blocchi $2 \times 2$ e quindi bisogna mol-

tiplicare sette $\frac{n}{2} \times \frac{n}{2}$ matrici. Applicando ricorsivamente l'algoritmo richiede in totale $O(n^{\log_2 7} \approx O(n^{2.807})$ operazioni sul campo.

La stabilità è inferiore rispetto all'algoritmo naive, ma più veloce nel caso in cui $n > 100$ circa.

continuare a scrivere...

# 3 Functions

## 3.1 Matrix norm

La norma di una matrice è una funzione $\|\cdot\| : \mathbb{K}^{n\times m} \mapsto \mathbb{K}$ che soddisfa le seguenti proprietà:

- $\|\mathbf{A}\| \geq 0$ con l'uguaglianza solamente se $\mathbf{A} = \mathbf{0}$ (sempre positiva)
- $\|\alpha\mathbf{A}\| = |\alpha|\ \|\mathbf{A}\|$ per ogni $\alpha \in \mathbb{K}$ (omogeneità)
- $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$ (disuguaglianza triangolare)

Si tratta di un operatore analogo alla norma di un vettore, ma applicata alle matrici.

Un'importante classe di norme matriciali è data dalla *subordinate matrix norm* definita a partire dalla norma di un vettore:

$$\|\mathbf{A}\|_p = \max_{\mathbf{x}\neq\mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \tag{3}$$

che equivalentemente può essere scritta come $\|\mathbf{A}\|_p = \max_{\|\mathbf{x}\|_p=1} \|\mathbf{A}\mathbf{x}\|_p$. Per le norme 1–, 2– e $\infty$– del vettore si ha che:

$$\|\mathbf{A}\|_1 = \max_{1\leq j\leq m} \sum_{i=1}^{n} |A_{ij}| \tag{4}$$

$$\|\mathbf{A}\|_2 = \max_{1\leq i\leq n} \sum_{j=1}^{m} |A_{ij}| \tag{5}$$

$$\|\mathbf{A}\|_\infty = \sigma_{\max}(\mathbf{A}) \tag{6}$$

dove $\sigma_{\max}(\mathbf{A})$ denota il valore singolare maggiore di $\mathbf{A}$. per la forma generale della norma $p$-esima di Hölder non è presente una formula esplicita per i casi in cui $p \neq 1, 2, \infty$.

Un'altra norma importante è quella di Frobenius (usata ad esempio nella definizione dell'errore per la regressione lineare):

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{n}\sum_{j=1}^{m}|A_{ij}^2|} = \sqrt{\mathrm{tr}(\mathbf{A}\mathbf{A}^T)} \tag{7}$$

# 4 Gauss elimination

L'eliminazione di Gauss consiste nell'eseguire operazioni sulla matrice che non cambiano il risultato del sistema di equazioni associato, come ad esempio *scambiare delle righe, moltiplicare righe per scalari e sommare una riga all'altra*. Queste operazioni sono eseguite per semplificare il problema.

L'eliminazione di Gauss viene usata per portare la matrice ad una forma più facile da gestire, chiamata *Row Echelon Form* (REF). In questa forma la matrice ha le seguenti proprietà:

- il primo[2] elemento non nullo di una riga (*pivot*) è esattamente 1.0
- righe che hanno tutti elementi nulli sono sotto le righe che hanno almeno un elemento non nullo
- ogni pivot di una riga si trova ad una colonna alla destra dei pivot delle righe superiori

L'algoritmo che trasforma la matrice nella *Row Echelon Form* è il seguente:

1. Trova il pivot (primo elemento non nullo della riga). Se la colonna ha elementi tutti nulli passa alla colonna successiva.
2. Scambia le righe posizionando la riga del pivot come prima.
3. Moltiplica ogni elemento sulla riga per $\frac{1}{\text{pivot}}$ in maniera tale che il pivot è pari a 1.0.
4. Tramite la somma tra righe (premoltiplicando per il pivot dell'altra riga), fa in maniera tale che tutti gli elementi sulla colonna del pivot (elementi che non sono il pivot) sono pari a zero.
5. Continua il processo finché non si è giunti alla *Row Echelon Form*.

La forma *Reduced Row Echelon Form* (RREF) consiste nell'avere solamente un valore diverso da zero per ogni colonna. Si nota che una matrice può avere molte REF, ma solamente una RREF.

[2]Primo elemento non nullo della riga partendo da sinistra verso destra.

## 4.1 Basic operations

Le operazioni base dell'eliminazione di Gauss sono:

- Scambio di due righe (row swapping: `matd_row_swap_r`).
- Moltiplica ogni elemento della riga per uno scalare diverso da zero (scalar multiplication of rows: `matd_row_smul_r`).
- Moltiplica una riga per in termine diverso da zero e aggiunge il risultato a un'altra riga (row addition: `matd_row_addrow_r`).

### 4.1.1 Row swapping

```c
int matd_row_swap_r(matd *m, unsigned int row1, unsigned int row2)
{
        // swap two rows of matrix m
        int i;
        double tmp;
        if(row1 >= m->n_rows || row2 >= m->n_rows) return 0; // cannot swap rows
        for(i=0; i<m->n_cols; i++){
                tmp = m->data[row2*m->n_cols+i];
                m->data[row2*m->n_cols+i] = m->data[row1*m->n_cols+i];
                m->data[row1*m->n_cols+i] = tmp;
        }
        return 1;
}
```

Esempio:

$$\texttt{matd_row_swap_r} = \left(\begin{bmatrix}1 & 2 & 3\\0 & 2 & 4\\2 & 1 & 9\end{bmatrix}, 0, 1\right) = \begin{bmatrix}0 & 2 & 4\\1 & 2 & 3\\2 & 1 & 9\end{bmatrix}$$

### 4.1.2 Scalar multiplication of rows

```c
int matd_row_smul_r(matd *m, unsigned int row, double num)
{
        int i;
        if(row >= m->n_rows) return 0; // cannot row multiply
        for(i=0; i<m->n_cols; i++) m->data[row*m->n_cols+i] *= num;
        return 1;
}
```

Esempio:

$$\texttt{matd_row_swap_r} = \left(\begin{bmatrix}1 & 2 & 3\\0 & 2 & 4\\2 & 1 & 9\end{bmatrix},\ 1,\ 2.0\right) = \begin{bmatrix}1 & 2 & 3\\0.0 & 4.0 & 6.0\\2 & 1 & 9\end{bmatrix}$$

### 4.1.3 Row addition

```c
int matd_row_addrow_r(matd *m, unsigned int where, unsigned int row,
    double multiplier)
{
        int i = 0;
        if(where >= m->n_rows || row >= m->n_rows) return 0; //
            cannot add rows
        for(i=0; i<m->n_cols; i++) m->data[where*m->n_cols+i] +=
            multiplier * m->data[row*m->n_cols+i];
        return 1;
}
```

Esempio:

$$\begin{aligned}\texttt{matd_row_addrow_r} &= \left(\begin{bmatrix}1 & 2 & 3\\0 & 2 & 4\\2 & 1 & 9\end{bmatrix},\ 0,\ 1,\ 0.5\right) = \\ &= \begin{bmatrix}1+0\cdot 0.5 & 2+2\cdot 0.5 & 3+4\cdot 0.5\\0 & 2 & 4\\4 & 1 & 9\end{bmatrix} = \\ &= \begin{bmatrix}1 & 3 & 5\\0 & 2 & 4\\2 & 1 & 9\end{bmatrix}\end{aligned}$$

# 5 Matrix factorizations

La fattorizzazione, o la decomposizione, di una matrice consiste nel *dividere la matrice nel prodotto di matrici più semplici*. In letteratura sono presenti varie decomposizioni applicabili a diverse classi di problemi:

**Decomposizione LU:** La matrice $\mathbf{A} = \mathbf{LU}$ è decomposta in una matrice triangolare inferiore $\mathbf{L}$ e una triangolare superiore $\mathbf{U}$. Applicabile a matrici quadrate.

**Decomposizione QR:** La matrice $\mathbf{A} = \mathbf{QR}$ é divisa in una matrice $\mathbf{Q}$ ortogonale e una $\mathbf{R}$ triangolare superiore di dimensione $n \times m$. Applicabile a matrici di dimensione qualsiasi $n \times m$.

**Decomposizione di Cholesky:** La matrice $\mathbf{A} = \mathbf{U}^T\mathbf{U}$ é scomposta tramite una matrice $\mathbf{U}$ triangolare superiore con elementi sulla diagonale positivi. Si applica a matrici quadrate, simmetriche, definite positive.

**Decomposizione a valori singolari:** La matrice $\mathbf{A} = \mathbf{UDV}^H$ é scomposta in una matrice $\mathbf{D}$ diagonale non-negativa e due matrici unitarie $\mathbf{U}$ e $\mathbf{V}$ ($\mathbf{V}^H$ denota la trasposta coniugata di $\mathbf{V}$). Applicabile a matrici di dimensione $n \times m$.

**Decomposizione spettrale:** La matrice $\mathbf{A} = \mathbf{VDV}^{-1}$ viene decomposta tramite la matrice $\mathbf{V}$ le cui colonne sono gli autovettori di $\mathbf{A}$ e la matrice $\mathbf{D}$ é diagonale con i corrispondenti autovalori come diagonale. Applicabile a matrici quadrate con autovettori distinti[3].

**Decomposizione di Schur:**

**Decomposizione in componenti principali:**

**Decomposizione non-negativa:** La matrice $\mathbf{A} = \mathbf{WH}$ è decomposta in due matrici che possiedono elementi positivi. É una soluzione, in genere di minimo locale, della funzione obiettivo $f(\mathbf{W}, \mathbf{H}) = \frac{1}{2}\|\mathbf{A} - \mathbf{WH}\|_F^2$ con i vincoli $W_{ij} \geq 0$ e $H_{hk} \geq 0$. Applicabile a matrici $n \times m$ non-negative.

## 5.1 LU(P) decomposition

La decomposizione LU, chiamata anche la fattorizzazione LU, si riferisce alla scomposizione di una matrice $\mathbf{A}$ in due matrici $\mathbf{L}$ e $\mathbf{U}$:

$$\mathbf{A} = \mathbf{L}\,\mathbf{U} \qquad \rightarrow \qquad \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ L_{21} & 1 & 0 \\ L_{31} & L_{32} & 1 \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix} \tag{8}$$

In pratica questa fattorizzazione viene ricavata da operazioni elementari sulle righe (come lo scambio delle righe). In tal caso bisogna introdurre nell'equazione una matrice di permutazione $\mathbf{P}$ che tiene conto del cambio delle righe:

$$\mathbf{P}\,\mathbf{A} = \mathbf{L}\,\mathbf{U} \tag{9}$$

[3]non necessariamente anche distinti autovalori.

dove **P** è una matrice ricavata dalla permutazione delle righe di una matrice identità **I**, ed è calcolata dalla procedura che trova **L** e **U**. Se una matrice **A** é quadrata (di dimensione $n \times n$) può sempre essere scomposta come $\mathbf{P}\ \mathbf{A} = \mathbf{L}\ \mathbf{U}$. Per trovare la decomposizione LU si usa versione modificata dell'algoritmo per l'eliminazione di Gauss. Questa implementazione richiede circa $\frac{2}{3}n^3$ operazioni a virgola mobile. Altri algoritmi richiedono funzioni ricorsive o randomizzazione.

Trovare la decomposizione (9) permette di calcolare il determinante della matrice **A**, la sua inversa e, quindi, anche risolvere i sistemi di equazioni lineari.

Esiste anche un'altra fattorizzazione dove non solo le righe, ma anche le colonne vengono considerate e si chiama *LU Factorization with full pivoting*.

Descrizione dell'algoritmo e codice per la soluzione tramite decomposizione LU

Aggiungere quello che dicono su Wikipedia

## 5.2 QR decomposition

Qualsiasi matrice simmetrica **A** può essere decomposta come:

$$\mathbf{A} = \mathbf{Q}\ \mathbf{R} \tag{10}$$

dove **Q** è una matrice ortogonale[4] e **R** una matrice triangolare superiore. Per trovare la decomposizione (10) si usa l'algoritmo di Gram-Schmidt sicuro?.

Si può dimostrare che tutte le matrici quadrate ammettono una decomposizione QR, anche se essa non è unica. Nel caso in cui la matrice **A** sia a coefficienti complessi, allora **Q** è una matrice unitaria.

La fattorizzazione QR di una matrice richiede un tempo proporzionale a $O(n^3)$ (tramite trasformazioni di Householder o quelle di Givens). Nonostante la complessità computazionale è simile a quella della decomposizione LU (o algoritmo di Gauss) questo risulta avere una migliore stabilità numerica. Inoltre la fattorizzazione QR può essere utilizzata per il calcolo di basi ortonormali e per la risoluzione di un sistema ai minimi quadrati. La decomposizione QR viene usata anche nel *metodo QR* utilizzato per calcolare gli autovalori e autovettori di una matrice.

Al contrario della fattorizzazione LU(P) che si concentra sulle operazioni sulle righe, la decomposizione QR utilizza operazioni sulle colonne.

[4] Una matrice si dice *ortogonale* se

$$\mathbf{Q}\ \mathbf{Q}^T = \mathbf{Q}^T\mathbf{Q} = \mathbf{I} \tag{11}$$

Questo equivale a dire che la trasposta della matrice è uguale all'inversa ($\mathbf{Q}^T = \mathbf{Q}^{-1}$).

Considerando le matrici (in questo caso di dimensione $3 \times 3$):

$$\mathbf{A} = \begin{bmatrix} | & | & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \mathbf{a}_3 \\ | & | & | \end{bmatrix} \qquad \mathbf{Q} = \begin{bmatrix} | & | & | \\ \mathbf{q}_1 & \mathbf{q}_2 & \mathbf{q}_3 \\ | & | & | \end{bmatrix} \tag{12}$$

dove $\mathbf{a}_i$ sono i vettori colonna che formano la matrice $\mathbf{A}$ e $\mathbf{q}_i$ i vettori colonna di $\mathbf{B}$. La struttura della matrice $\mathbf{R}$ deriva dal fatto che $\mathbf{Q}$, essendo una matrice ortogonale, ha per colonne dei vettori unitari, ovvero di norma euclidea unitaria. Questo porta alle seguenti formule:

$$\begin{cases} \mathbf{q}_1 = \dfrac{\mathbf{a}_1}{\|\mathbf{a}_1\|} \\ \mathbf{q}_2 = \dfrac{\mathbf{a}_2^\perp}{\|\mathbf{a}_2^\perp\|} \\ \mathbf{q}_2 = \dfrac{\mathbf{a}_3^\perp}{\|\mathbf{a}_3^\perp\|} \end{cases} \qquad \text{dove} \qquad \begin{aligned} \mathbf{a}_2^\perp &= \mathbf{a}_2 - \langle \mathbf{a}_2, \mathbf{q}_1 \rangle \, \mathbf{q}_1 \\ \mathbf{a}_3^\perp &= \mathbf{a}_3 - \langle \mathbf{a}_3, \mathbf{q}_1 \rangle \, \mathbf{q}_1 - \langle \mathbf{a}_3, \mathbf{q}_2 \rangle \, \mathbf{q}_2 \end{aligned} \tag{13}$$

dove $\langle \mathbf{u}, \mathbf{v} \rangle$ è il prodotto scalare tra i due vettori $\mathbf{u}$ e $\mathbf{v}$, mentre $\|\mathbf{u}\|$ è la norma-2 euclidea. Quindi la decomposizione QR di questo esempio per matrici $3 \times 3$ è pari a:

$$\mathbf{A} = \begin{bmatrix} | & | & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \mathbf{a}_3 \\ | & | & | \end{bmatrix} = \begin{bmatrix} | & | & | \\ \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|} & \frac{\mathbf{a}_2^\perp}{\|\mathbf{a}_2^\perp\|} & \frac{\mathbf{a}_3^\perp}{\|\mathbf{a}_3^\perp\|} \\ | & | & | \end{bmatrix} \begin{bmatrix} \|\mathbf{a}_1\| & \langle \mathbf{a}_2, \mathbf{q}_1 \rangle & \langle \mathbf{a}_3, \mathbf{q}_1 \rangle \\ 0 & \|\mathbf{a}_2^\perp\| & \langle \mathbf{a}_3, \mathbf{q}_2 \rangle \\ 0 & 0 & \|\mathbf{a}_3^\perp\| \end{bmatrix} \tag{14}$$

Questa formula può essere generalizzata al caso $n \times n$.

## 5.3 Cholesky decomposition

## 5.4 Singular value decomposition (SVD)

## 5.5 Spectral decomposition

## 5.6 Schur decomposition

## 5.7 Principal component analysis (PCA)

## 5.8 Non-negative factorization

# 6 Solve linear systems

Risolvere il sistema di equazioni lineari:

$$\mathbf{A}\,\mathbf{x} = \mathbf{b} \tag{15}$$

vuol dire trovare il vettore colonna incognito $\mathbf{x}$ sapendo la matrice $\mathbf{A}$ e il vettore dei termini noti $\mathbf{b}$. La soluzione è trovata in maniera esatta tramite la matrice inversa di $\mathbf{A}$:

$$\mathbf{x} = \mathbf{A}^{-1}\,\mathbf{b} \tag{16}$$

Dal punto di vista dell'implementazione il principale problema risiede nel fatto che calcolare la matrice inversa$\mathbf{A}^{-1}$ attraverso il metodo di Come si chiamava? (quello lentissimo esatto) è un'operazione molto dispendiosa. Per questo esistono molti metodi per calcolare velocemente il vettore ignoto $\mathbf{x}$ in maniera computazionalmente efficiente.

## 6.1 Gauss-Jordan elimination

### 6.1.1 Backward substitution

La sostituzione all'indietro è la tecnica per la quale si calcola la soluzione del sistema lineare $\mathbf{U}\mathbf{x} = \mathbf{b}$ dove $\mathbf{U}$ è una matrice triangolare superiore:

$$\mathbf{U}\,\mathbf{x} = \mathbf{b} \qquad \rightarrow \qquad \begin{bmatrix} U_{11} & U_{12} & \cdots & U_{1n} \\ 0 & U_{22} & \cdots & U_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & U_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \tag{17}$$

Si noti che l'ultimo termine del vettore $\mathbf{x}$ è facilmente calcolabile dall'ultima equazione (quella corrispondente alla $n$-esima riga). Questa equazione

permette di trovare $x_n$ e poi si procede *all'indietro* per trovare le altre componenti del vettore delle incognite $\mathbf{x}$. Per questo motivo è chiamato *metodo della sostituzione all'indietro*.

$$x_n = \frac{b_n}{U_{nn}} \qquad x_{n-1} = \frac{b_{n-1} - U_{(n-1)n}x_n}{U_{(n-1)(n-1)}} \qquad x_i = \frac{b_i - \sum_{k=i}^{n-1} U_{ik}x_k}{U_{ii}} \tag{18}$$

```
matd *lu_solvebck(matd *U, matd *b)
{
        matd *x = new_matd(U->n_cols, 1);
        int i = U->n_cols, j;
        double tmp;
        while(i-- > 0){
                tmp = b->data[i*b->n_cols];
                for(j=i; j<U->n_cols; j++) tmp -= U->data[i*U->n_cols
                    +j] * x->data[j*x->n_cols];
                x->data[i*x->n_cols] = tmp / U->data[i*U->n_cols+i];
        }
        return x;
}
```

### 6.1.2 Forward substitution

Questo è il caso opposto alla sostituzione all'indietro, ovvero un metodo per trovare la soluzione del sistema lineare $\mathbf{Lx} = \mathbf{b}$ dove $\mathbf{L}$ è, al contrario del caso precedente, una matrice triangolare inferiore.

$$\mathbf{L}\,\mathbf{x} = \mathbf{b} \qquad \rightarrow \qquad \begin{bmatrix} L_{11} & 0 & \cdots & 0 \\ L_{21} & L_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ L_{n1} & L_{n2} & \cdots & L_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \tag{19}$$

La sostituzione in avanti risulta essere, quindi, pari a:

$$x_1 = \frac{b_1}{L_{11}} \qquad x_2 = \frac{b_2 - L_{21}x_1}{L_{22}} \qquad x_i = \frac{b_i - \sum_{k=1}^{i-1} L_{ik}x_k}{L_{ii}} \tag{20}$$

```
matd *lu_solvefwd(matd *L, matd *b)
{
        matd *x = new_matd(L->n_cols, 1);
        int i,j;
        double tmp;
        for(i=0; i<L->n_cols; i++){
```

```
                tmp = b->data[i*b->n_cols];
                for(j=0; j<i; j++){
                        tmp -= L->data[i*L->n_cols+j] * x->data[j*x->
                            n_cols];
                }
                x->data[i*x->n_cols] = tmp / L->data[i*L->n_cols+i];
        }
        return x;
}
```

## 6.2 LU(P) decomposition

Usando la decomposizione (9) il sistema di equazioni (15) diventa:

$$\mathbf{A}\,\mathbf{x} = \mathbf{b} \qquad \rightarrow \qquad \mathbf{P}\,\mathbf{A}\,\mathbf{x} = \mathbf{P}\,\mathbf{b} \qquad \rightarrow \qquad \mathbf{L}\;\mathbf{U}\,\mathbf{x} = \mathbf{P}\,\mathbf{b} \tag{21}$$

Introducendo il sistema di equazioni ausiliario $\mathbf{y} = \mathbf{U}\,\mathbf{x}$ troviamo la soluzione $\mathbf{x}$ del sistema lineare (15) risolvendo i due sistemi:

$$\mathbf{y} = \mathbf{U}\,\mathbf{x} \tag{22}$$

$$\mathbf{L}\,\mathbf{y} = \mathbf{P}\,\mathbf{b} \tag{23}$$

Per risolvere questi due sistemi di equazioni lineari si procede a trovare il vettore temporaneo $\mathbf{y}$ tramite sostituzione in avanti (*forward substitution*) del sistema $\mathbf{L}\;\mathbf{y} = \mathbf{P}\;\mathbf{b}$, dato che $\mathbf{L}$ è una matrice triangolare inferiore. Successivamente si trova la soluzione $\mathbf{x}$ tramite il sistema $\mathbf{U}\;\mathbf{x} = \mathbf{y}$ tramite sostituzione all'indietro[5] (*backward substitution*), dato che la matrice $\mathbf{U}$ è triangolare superiore.

```
matd *lu_solve(matd_lup *lu, matd* b)
{
        matd *Pb, *x, *y;
        if(lu->U->n_rows != b->n_rows || b->n_cols != 1) return NULL;
        Pb = matd_mul(lu->P, b);

        y = lu_solvefwd(lu->L, Pb); // Solve L*y = P*b using forward
            substition
        x = lu_solvebck(lu->U, y); // Solve U*x=y using backward
            substitution

        free_mat(y); // free memory space of the temporary matrices
```

[5] All'indietro, nel senso che parte trovando l'ultimo termine di $\mathbf{x}$ e va all'indietro fino al primo $x_1$. Il contrario rispetto alla *forward substitution*.

```
        free_mat(Pb);
        return x;
}
```

## 6.3 QR decomposition

Fattorizzata la matrice $\mathbf{A} = \mathbf{QR}$ di un sistema lineare $\mathbf{Ax} = \mathbf{b}$ la soluzione di tale sistema è data da:

$$\mathbf{x} = \mathbf{R}^{-1}(\mathbf{Q}^T\mathbf{b}) \tag{24}$$

Il calcolo del vettore temporaneo $\mathbf{y} = \mathbf{Q}^T\mathbf{b}$ richiede $O(n^2)$ operazioni per la moltiplicazione matrice-vettore, mentre il calcolo di $\mathbf{x} = \mathbf{R}^{-1}\mathbf{y}$ si può eseguire attraverso un algoritmo di sostituzione all'indietro con un tempo computazionale sempre di $O(n^2)$ operazioni. Il costo computazionale maggiore, dunque, è quello della fattorizzazione (pari a $O(n^3)$ operazioni).

. . . . . .

2. Il Collo di Bottiglia: L'incubo di malloc nella QR

Questa è la cosa più importante da migliorare per le performance. Nella tua funzione `mat_qr_solve`, ci sono queste righe dentro ai cicli `for` annidati:

```
mat *q_j = mat_get_col(Q, j);
TYPE r_ji = mat_dot(q_j, v_i);
mat *proj = mat_mul_scalar(q_j, r_ji);
mat *v_i_new = mat_sub(v_i, proj);
mat_free(v_i);
v_i = v_i_new;
mat_free(q_j);
mat_free(proj);
```

Il codice fa esattamente ciò che deve fare matematicamente, e gestisce bene la memoria liberandola. Ma in MS-DOS, chiamare `malloc` e `free` (tramite `mat_get_col`, `mat_mul`, ecc.) migliaia di volte al secondo in un ciclo interno distruggerà le performance e frammenterà l'heap fino al blocco totale.

Soluzione (QR "In-Place"): Possiamo fare i calcoli matematici leggendo e scrivendo direttamente i numeri nell'array `data`, senza allocare mai nessun vettore intermedio. Sostituisci l'intero contenuto di `mat_qr_solve` con questo. È spaventosamente più veloce:

```
mat_qr* mat_qr_solve_Gemini(const mat *A)
{
        mat_qr *qr = (mat_qr*)malloc(sizeof(mat_qr));
        int i, j, r;
        TYPE norm, dot;

```

```
        qr->Q = mat_copy(A);          // Q inizia come copia esatta di A
        qr->R = mat_new(A->n_cols, A->n_cols);

        for(i = 0; i < A->n_cols; i++) {
                // 1. Calcola la norma della colonna i di Q
                norm = 0.0;
                for(r = 0; r < A->n_rows; r++) {
                        norm += qr->Q->data[r * qr->Q->n_cols + i] * qr->Q->data[r * qr->Q->n_cols + i];
                }
                norm = sqrt(norm);
                qr->R->data[i * qr->R->n_cols + i] = norm;

                if (norm < SLAP_MIN_COEF) continue; // Evita divisioni per zero

                // 2. Normalizza la colonna i di Q
                for(r = 0; r < A->n_rows; r++) {
                        qr->Q->data[r * qr->Q->n_cols + i] /= norm;
                }

                // 3. Ortogonalizza le colonne successive (Gram-Schmidt Modificato)
                for(j = i + 1; j < A->n_cols; j++) {
                        dot = 0.0;
                        // Prodotto scalare tra colonna i e colonna j
                        for(r = 0; r < A->n_rows; r++) {
                                dot += qr->Q->data[r * qr->Q->n_cols + i] * qr->Q->data[r * qr->Q->n_cols + j];
                        }
                        qr->R->data[i * qr->R->n_cols + j] = dot;

                        // Sottrai la proiezione dalla colonna j
                        for(r = 0; r < A->n_rows; r++) {
                                qr->Q->data[r * qr->Q->n_cols + j] -= dot * qr->Q->data[r * qr->Q->n_cols + i];
                        }
                }
        }
        return qr;
}
```

## 6.4 Conjugate gradient method

Il metodo del gradiente coniugato in matematica serve per trovare iterativamente la soluzione di sistemi lineari definiti da una matrice positiva. Spesso è usato per matrici definite positive sparse che hanno una dimensione eccessiva per poterli risolvere convenientemente con metodi diretti come la decomposizione di Cholesky. Il metodo del gradiente coniugato è usato anche per risolvere problemi di ottimizzazione non-condizionati.

Il metodo del gradiente bi-coniugato provvede la generalizzazione a matrici non-simmetriche. Esistono anche metodi del gradiente coniugato non-lineari.

Supponiamo di avere un sistema lineare (15) con vettore $\mathbf{x}$ di incognite di dimensione $n$ e con la matrice $\mathbf{A} \in \mathbb{K}^{n \times n}$ simmetrica, ovvero con $\mathbf{A}^T = \mathbf{A}$, definita positiva, ovvero con $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$ per ogni vettore non-nullo $\mathbf{x} \in \mathbb{K}^n$ e reale.

Questo metodo può essere visto come una variazione del metodo iterativo di Arnoldi/Lanczos per il problema agli autovalori.

Diciamo che due vettori $\mathbf{u}$ e $\mathbf{v}$ sono coniugati rispetto a $\mathbf{A}$ se:

$$\mathbf{u}^T \mathbf{A} \mathbf{v} = 0 \tag{25}$$

Dal momento che $\mathbf{A}$ è simmetrica e definita positiva, la parte a sinistra dell'uguale definisce un prodotto interno:

$$\mathbf{u}^T \mathbf{A} \mathbf{v} = \langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{A}} := \langle \mathbf{A}\mathbf{u}, \mathbf{v} \rangle = \left\langle \mathbf{u}, \mathbf{A}^T \mathbf{v} \right\rangle = \langle \mathbf{u}, \mathbf{A}\mathbf{v} \rangle \tag{26}$$

Due vettori sono coniugati rispetto ad $\mathbf{A}$ se e solo se sono ortogonali rispetto a questo prodotto interno (se il prodotto interno è pari a zero).

Supponiamo che $P = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n\}$ è un insieme di $n$ vettori mutuamente coniugati rispetto a $\mathbf{A}$, ovvero che $\mathbf{p}_i^T \mathbf{A} \mathbf{p}_j = 0$ per ogni $i \neq j$. Allora $P$ forma una base di $\mathbb{K}^n$ ed esprimiamo le soluzioni $\mathbf{x}^*$ del sistema $\mathbf{A}\mathbf{x} = \mathbf{b}$ in questa base:

$$\mathbf{x}^* = \sum_{i=1}^{n} \alpha_i \mathbf{p}_i \qquad \rightarrow \qquad \mathbf{A}\mathbf{x}^* = \sum_{i=1}^{n} \alpha_i \mathbf{A}\mathbf{p}_i \tag{27}$$

Moltiplicando a sinistra il sistema $\mathbf{A}\mathbf{x} = \mathbf{b}$ con il vettore $\mathbf{p}_k^T$ si ottiene:

$$\mathbf{p}_k^T \mathbf{b} = \mathbf{p}_k^T \mathbf{A} \mathbf{x}^* = \sum_{i=1}^{n} \alpha_i \mathbf{p}_k^T \mathbf{A} \mathbf{p}_i = \sum_{i=1}^{n} \alpha_i \langle \mathbf{p}_k, \mathbf{p}_i \rangle_{\mathbf{A}} = \alpha_k \langle \mathbf{p}_k, \mathbf{p}_k \rangle_{\mathbf{A}} \tag{28}$$

e quindi otteniamo:

$$\alpha_k = \frac{\langle \mathbf{p}_k, \mathbf{b} \rangle}{\langle \mathbf{p}_k, \mathbf{p}_k \rangle} \tag{29}$$

Quindi il metodo per risolvere il sistema (15) consiste nel trovare una sequenza $P$ di $n$ direzioni coniugate e poi calcolare i coefficienti $\alpha_k$.

Se scegliamo i vettori $\mathbf{p}_k$ adeguatamente, allora non necessitiamo neanche di tutti i vettori per avere una approssimazione soddisfacente della soluzione $\mathbf{x}$.

Definiamo il valore iniziale di $\mathbf{x}^*$ come $\mathbf{x}_0$ (inizialmente possiamo porre $\mathbf{x}_0 = \mathbf{0}$). La funzione obiettivo da minimizzare è la seguente forma quadratica:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} - \mathbf{x}^T\mathbf{b} \tag{30}$$

Il gradiente (derivata prima) è pari a:

$$\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} \tag{31}$$

mentre la derivata seconda è pari alla matrice Hessiana $\mathbf{H}(f(\mathbf{x})) = \mathbf{A}$ che per definizione della matrice $\mathbf{A}$ è anch'essa definita positiva.

Questo suggerisce a prendere come vettore $\mathbf{p}_0$ il negativo del gradiente di $f$ a $\mathbf{x} = \mathbf{x}_0$. Questo vuol dire che $\mathbf{p}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$. Gli altri vettori saranno coniugati al gradiente (da qui il nome del metodo). Si noti inoltre che $\mathbf{p}_0$ è anche il residuo calcolato dal primo step dell'algoritmo. Sia $\mathbf{r}_k$ il residuo al $k$-esimo step (anche uguale al negativo del gradiente di $f$ al $k$-esimo step):

$$\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k \tag{32}$$

lo step successivo dovrà muoversi in direzione $\mathbf{r}_k$ dove le altre direzioni sono coniugate l'una all'altra. Una maniera pratica per assicurarsi questo è trovare le direzioni successive dal residuo corrente e da tutte le direzioni precedenti. Questo metodo di calcolare il vincolo può essere visto come un esempio dell'ortonormalizzazione di Gram-Schmidt:

$$\mathbf{p}_k = \mathbf{r}_k - \sum_{i<k} \frac{\mathbf{p}_i^T\mathbf{A}\mathbf{r}_k}{\mathbf{p}_i^T\mathbf{A}\mathbf{p}_i}\mathbf{p}_i \tag{33}$$

Seguendo questa direzione la seguente posizione ottimale è:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k\mathbf{p}_k \tag{34}$$

con

$$\alpha_k = \frac{\mathbf{p}_k^T(\mathbf{b} - \mathbf{A}\mathbf{x}_k)}{\mathbf{p}_k^T\mathbf{A}\mathbf{p}_k} = \frac{\mathbf{p}_k^T\mathbf{r}_k}{\mathbf{p}_k^T\mathbf{A}\mathbf{p}_k} \tag{35}$$

che deriva dall'aver posto pari a zero la derivata di $f(\mathbf{x}_{k+1})$ rispetto a $\alpha_k$.

Nell'algoritmo $\alpha_k$ è scelto in maniera tale che $\mathbf{t}_{k+1}$ è ortogonale a $\mathbf{r}_k$

Codice Matlab

**Algorithm 4** Conjugate gradient

**Require:** matrix $\mathbf{A}$, vector $\mathbf{b}$

$\mathbf{r}_0 \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}_0$
$\mathbf{p}_0 \leftarrow \mathbf{r}_0$
$k \leftarrow 0$
**while** $\mathbf{r}_k$ is sufficiently small **do**
    $\alpha_k \leftarrow \frac{\mathbf{p}_k^T \mathbf{r}_k}{\mathbf{p}_k^T \mathbf{A}\mathbf{p}_k}$
    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{p}_k$
    $\mathbf{r}_{k+1} \leftarrow \mathbf{r}_k - \alpha_k \mathbf{A}\mathbf{p}_k$
    $\beta_k \leftarrow \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$
    $\mathbf{p}_{k+1} \leftarrow \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$
    $k \leftarrow k + 1$
**end while**
return $\mathbf{x}_{k+1}$ as the result

```
function x = conjgrad(A, b, x)
    r = b - A * x;
    p = r;
    rsold = r' * r;

    for i = 1:length(b)
        Ap = A * p;
        alpha = rsold / (p' * Ap);
        x = x + alpha * p;
        r = r - alpha * Ap;
        rsnew = r' * r;
        if sqrt(rsnew) < 1e-10
            break
        end
        p = r + (rsnew / rsold) * p;
        rsold = rsnew;
    end
end
```

. . . . . .

SPIEGAZIONE DA GEMINI:

La Teoria del Metodo del Gradiente Coniugato (CG) Il metodo del Gradiente Coniugato è uno degli algoritmi iterativi più efficienti per la risoluzio-

ne di grandi sistemi di equazioni lineari della forma:

$$A\mathbf{x} = \mathbf{b}$$

Affinché il metodo converga e sia matematicamente ben definito, la matrice $A$ di dimensioni $n \times n$ deve essere Simmetrica ($A = A^T$) e Definita Positiva ($\mathbf{x}^T A\mathbf{x} > 0$ per ogni $\mathbf{x} \neq \mathbf{0}$). 1. Riformulazione come Problema di Ottimizzazione L'intuizione fondamentale alla base del Gradiente Coniugato è che risolvere il sistema lineare $A\mathbf{x} = \mathbf{b}$ equivale a trovare il punto di minimo di una specifica funzione quadratica. Definiamo la forma quadratica:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$$

Se calcoliamo il gradiente di questa funzione rispetto a $\mathbf{x}$, otteniamo:

$$\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$$

Il minimo della funzione si trova ponendo il gradiente uguale a zero, il che ci riporta esattamente al nostro sistema originale $A\mathbf{x} = \mathbf{b}$. Inoltre, in ogni punto $\mathbf{x}_k$, la direzione di massima pendenza (Steepest Descent) è l'opposto del gradiente, che coincide con il residuo del sistema lineare:

$$\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k = -\nabla f(\mathbf{x}_k)$$

2. Il Limite della Discesa del Gradiente (Steepest Descent) Nel classico metodo di discesa del gradiente, ci si muove sempre nella direzione del residuo $\mathbf{r}_k$. Tuttavia, questo approccio porta spesso a un fenomeno di "zig-zag" inefficiente: l'algoritmo finisce per percorrere più volte le stesse direzioni, specialmente se la matrice $A$ è malcondizionata (cioè se le linee di livello della funzione quadratica sono ellissi molto allungate). 3. Direzioni A-Ortogonali (Coniugate) Per evitare di ricalcolare correzioni lungo direzioni già esplorate, il Gradiente Coniugato introduce il concetto di direzioni coniugate (o $A$-ortogonali). Un insieme di vettori non nulli $\{\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_{n-1}\}$ si dice $A$-ortogonale se:

$$\mathbf{p}_i^T A\mathbf{p}_j = 0 \quad \text{per } i \neq j$$

Se scegliamo di muoverci lungo queste direzioni coniugate, l'algoritmo garantisce di trovare la soluzione esatta in al massimo $n$ passi (supponendo aritmetica esatta, senza errori di arrotondamento). 4. Le Equazioni dell'Algoritmo Ad ogni passo $k$, l'algoritmo aggiorna la soluzione $\mathbf{x}_k$ muovendosi lungo la direzione di ricerca $\mathbf{p}_k$ di una quantità $\alpha_k$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k\mathbf{p}_k$$

Il parametro $\alpha_k$ (la lunghezza del passo) è scelto per minimizzare $f(\mathbf{x})$ lungo la direzione $\mathbf{p}_k$, e si calcola come:

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}$$

Il nuovo residuo si aggiorna di conseguenza:

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$$

Il vero "trucco" magico del CG sta nel modo in cui calcola la nuova direzione $\mathbf{p}_{k+1}$. Invece di dover salvare tutti i vettori precedenti per applicare l'ortogonalizzazione di Gram-Schmidt, il CG richiede solo l'ultima direzione $\mathbf{p}_k$ e il nuovo residuo $\mathbf{r}_{k+1}$:

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

dove il coefficiente $\beta_k$ assicura che $\mathbf{p}_{k+1}$ sia $A$-ortogonale a $\mathbf{p}_k$:

$$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$$

Questa eleganza matematica si traduce in una straordinaria efficienza informatica: per ogni iterazione è sufficiente mantenere in memoria solo pochi vettori ($\mathbf{x}, \mathbf{r}, \mathbf{p}$) ed eseguire un'unica moltiplicazione matrice-vettore ($A\mathbf{p}_k$).

# 7 Determinante

## 7.1 Formula di Leibniz

il determinante di una matrice $\mathbf{A}$ di dimensione $n \times n$ è trovato tramite la seguente formula:

$$\det(\mathbf{A}) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} A_{i\,\sigma(i)} \tag{36}$$

dove $S_n$ è l'insieme di tutte le permutazioni $\sigma$ dell'insieme $1, 2, 3, \ldots, n$, mentre $\operatorname{sgn}(\sigma)$ denota il segno della permutazione (+1 se è una permutazione pari, -1 se è dispari). Questo metodo ha un elevato costo computazionale, pari a $O(N \cdot N!)$.

## 7.2 Sviluppo di Laplace

Questo metodo risulta efficace solamente se la matrice contiene un gran numero di zeri. Scegliendo una riga $i$ si calcola il determinante come:

$$\det(\mathbf{A}) = \sum_{j=1}^{n} A_{ij}\, C_{ij} \tag{37}$$

dove $C_{ij}$ è il complemento algebrico, ovvero è pari a $(-1)^{i+j}$ per il determinante del minore di ordine $n-1$ ottenuto dalla matrice $\mathbf{A}$ eliminando la riga $i$-esima e la colonna $j$-esima. Il metodo risulta efficiente in caso si scelga una riga $i$ che contiene il maggior numero di zeri possibile.

Lo stesso sviluppo può essere eseguito su una determinata colonna.

## 7.3 Algoritmo di Gauss

Nel caso di matrici triangolari (siano esse triangolari superiori o inferiori) il determinante è pari al prodotto degli elementi sulla diagonale principale. Questo deriva anche dallo sviluppo di Laplace.

## 7.4 Determinante tramite LU

La decomposizione LU calcola

$$\mathbf{P}\,\mathbf{A} = \mathbf{L}\,\mathbf{U}$$

dove $\mathbf{L}$ e' una matrice triangolare inferiore con ogni elemento sulla diagonale unitario e $\mathbf{U}$ una matrice triangolare superiore. Sappiamo che $\det(\mathbf{L}) = 1$ e che il determinante di una matrice triangolare superiore $\mathbf{U}$ è il prodotto della sua diagonale. Il segno dipende dal numero di permutazioni di $\mathbf{P}$. Calcolare il determinante è computazionalmente quasi gratuito: costa solo $O(N)$ operazioni aggiuntive rispetto alla decomposizione LU.

```
TYPE mat_det(const mat *m) {
    mat_lup *lu;
    TYPE det = 1.0;
    unsigned int i;

    if (m->n_rows != m->n_cols) return 0.0; // Solo matrici quadrate

    lu = mat_lup_solve(m);
    if (!lu) return 0.0; // Matrice singolare (non invertibile/
        degenere)
```

```

    for (i = 0; i < lu->U->n_rows; i++) {
        det *= lu->U->data[i * lu->U->n_cols + i];
    }

    // Se il numero di permutazioni delle righe e' dispari, il segno si inverte
    if (lu->num_permutations % 2 != 0) {
        det = -det;
    }

    mat_lup_free(lu);
    return det;
}
```

# 8 Inverse matrix

L'algoritmo peggiore in termini computazionali per trovare la matrice inversa fa uso della matrice aggiunta

$$\mathbf{A}^{-1} = \frac{\mathrm{adj}(\mathbf{A})}{\det(\mathbf{A})} \tag{38}$$

La matrice $\mathrm{adj}(\mathbf{A})$ è calcolata tramite la trasposta della matrice dei cofattori $\mathrm{adj}(\mathbf{A}) = \mathrm{cof}(\mathbf{A})^T$. La matrice dei cofattori $\mathrm{cof}(\mathbf{A})$ ha elementi:

$$\mathrm{cof}_{ij}(\mathbf{A}) = (-1)^{i+j} \det\left(\mathbf{M}_{ij}\right) \tag{39}$$

dove $\mathbf{M}_{ij}$ è la matrice minore di $\mathbf{A}$, ottenuta eliminando la $i$-esima riga e la $j$-esima colonna della matrice $\mathbf{A}$.

## 8.1 Tramite soluzione del sistema

Troviamo l'inversa della matrice $\mathbf{A}$ risolvendo il sistema

$$\mathbf{A}\mathbf{x}_i = \mathbf{e}_i \tag{40}$$

dove $\mathbf{e}_i$ e' il vettore della base canonica (ad esempio: $\mathbf{e}_1 = [1, 0, 0]$). In questo modo troviamo l'$i$-esima colonna $\mathbf{x}_i$ della matrice inversa $\mathbf{A}^{-1}$. Per risolvere il sistema (40) possiamo usare la decomposizione LU.

```
mat* mat_inv(const mat *m) {
    mat_lup *lu;
```

```c
    mat *inv, *e, *col_inv;
    unsigned int i, j;

    if (m->n_rows != m->n_cols) return NULL;

    lu = mat_lup_solve(m);
    if (!lu) return NULL; // Matrice singolare

    inv = mat_new(m->n_rows, m->n_cols);
    e = mat_new(m->n_rows, 1);

    for (i = 0; i < m->n_cols; i++) {
        // Genera il vettore colonna base e_i
        for (j = 0; j < m->n_rows; j++) {
            e->data[j] = (i == j) ? 1.0 : 0.0;
        }

        // Risolve il sistema per trovare la colonna i-esima dell'
            inversa
        col_inv = solve_lu(lu, e);

        if (col_inv) {
            // Copia la soluzione nel posto giusto all'interno di inv
            for (j = 0; j < m->n_rows; j++) {
                inv->data[j * inv->n_cols + i] = col_inv->data[j];
            }
            mat_free(col_inv);
        }
    }

    mat_free(e);
    mat_lup_free(lu);
    return inv;
}
```

Per risolvere il sistema (40) e' stato usato il metodo LU.

# 9 Eigen

| metodo | si applica a | produce | costo | conv. | descriz |
|---|---|---|---|---|---|
| Lanczos algorithm | Hermitian | $m$ largest/smallest eigenpair | | | |
| Power iteration | general | eigenpair with largest value | $O(n^2)$ | linear | applica |
| Inverse iteration | general | eigenpair with value closest to $\mu$ | | linear | power i |
| Rayleigh quotient iteration | hermitian | any eigenpair | | cubica | power i |
| Preconditioned inverse iteration | Hermitiana | any eigenpair | | cubic | inverse |
| Bisection method | real symmetric tridiagonal | any eigenvalue | | lineare | usa il m |
| Laguerre iteration | real symmetric tridiagonal | any eigenvalue | | cubic | metodo |
| QR algorithm | Hassenberg | all eigenvalues and all eigenpairs | $O(n^2)$ e $6n^3 + O(n^2)$ | cubic | applica |

## 9.1 QR algorithm

L'algoritmo più affidabile e maggiormente utilizzato è quello inventato da John G.F. Francis alla fine degli anni '60 che sfrutta la decomposizione QR.

L'algoritmo parte dalla matrice $\mathbf{A}$ di cui vogliamo calcolare gli autovalori e iterativamente arriva a triangolarizzarla. Allo step iniziale $k = 0$ la matrice $\mathbf{A}_0 = \mathbf{A}$ è posta pari alla matrice di input. Al generico step $k$ si procede alla decomposizione QR $\mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k$. Quindi allo step successivo la matrice $\mathbf{A}_{k+1}$ è trovata tramite:

$$\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k = \mathbf{Q}_k^{-1}\mathbf{Q}_k\mathbf{R}_k\mathbf{Q}_k = \mathbf{Q}_k^{-1}\mathbf{A}_k\mathbf{Q}_k = \mathbf{Q}_k^T\mathbf{A}_k\mathbf{Q}_k \tag{41}$$

Notare che tutte le matrici $\mathbf{A}_k$ sono simili e quindi hanno sempre gli stessi autovalori. L'algoritmo è numericamente stabile perché procede tramite trasformazioni simili ortogonali. Sotto certe condizioni le matrici $\mathbf{A}_k$ convergono ad una matrice triangolare, la *forma di Schur* di $\mathbf{A}$ (gli autovalori sono gli elementi sulla diagonale).

In questa forma grezza il calcolo delle iterazioni può essere computazionalmente oneroso e questo può essere ridotto trasformando inizialmente la matrice **A** in una forma di Hessenberg superiore. Questa trasformazione costa $\frac{10}{9}n^3 + O(n^2)$ operazioni aritmetiche usando una riduzione di Householder. Il calcolo della decomposizione QR di una matrice di Hessenberg superiore ha un costo di $6n^2 + O(n)$. Inoltre essendo la forma di Hessenberg una sorta di triangolare superiore questo riduce anche il numero di iterazioni per arrivare a convergenza.

```
mat* eigen_qr(const mat *m, TYPE tolerance, int max_iter)
{
        mat *A = mat_copy(m);
        mat_qr *qr;
        int iter, r, c;
        short converged;

        if(m->n_rows != m->n_cols) {
        mat_free(A);
        return NULL; // ERROR!! nxn square matrix
    }

        for(iter = 0; iter < max_iter; iter++){
                qr = mat_qr_solve_Gemini(A); // Usa la nostra versione MGS per maggiore stabilita'!
                mat_free(A); // avoid memory leakage
                A = mat_mul(qr->R, qr->Q); // update A matrix for i-th step
                mat_qr_free(qr); // free used memory for QR decomposition

                // --- NUOVO CONTROLLO DI CONVERGENZA ---
                converged = 1; // Assumiamo sia convergente

                // Esaminiamo solo la parte SOTTO la diagonale principale
                for(c = 0; c < A->n_cols - 1; c++) {
                        for(r = c + 1; r < A->n_rows; r++) {
                                if(fabs(A->data[r * A->n_cols + c]) > tolerance) {
                                        converged = 0; // Trovato un elemento troppo grande, non ha ancora finito!
                                        break;
                                }
                        }
                        if(!converged) break;
                }

                if(converged) {
```

```
                        #if SLAP_DEBUG
                        printf("QR␣Autovalori␣convergenza␣raggiunta␣
                            in␣%d␣iterazioni.\n", iter);
                        #endif
                        break;
                }
        }

        mat *diag = mat_get_diag(A); // Estrae gli autovalori
        mat_free(A);
        return diag;
}
```

A ogni iterazione calcolata di $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k$, controlla gli elementi sotto la diagonale principale. Se il valore assoluto di $A(i, i-1)$ scende sotto una tolleranza (il tuo `SLAP_ALMOST_ZERO`), considera quell'elemento zero. Quando tutti gli elementi sotto-diagonali sono zero, hai gli autovalori sulla diagonale.

Nota bene: L'algoritmo QR puro (senza shift) converge infallibilmente ad una matrice diagonale per matrici simmetriche. Per matrici non-simmetriche che possiedono autovalori complessi, potresti notare che l'algoritmo non convergerà mai del tutto e si arresterà al `max_iter`, poiché gli autovalori complessi in C reale si manifestano come blocchi $2 \times 2$ sulla diagonale che non vanno a zero.

## 9.2 Metodo delle potenze

Il metodo delle potenze serve a trovare l'autovalore dominante (quello con valore assoluto maggiore) e il suo corrispondente autovettore. È un algoritmo iterativo.

Si parte da un vettore casuale $\mathbf{x}$, e lo si moltiplica ripetutamente

$$\mathbf{x}_{k+1} = \frac{\mathbf{A}\mathbf{x}_k}{||\mathbf{A}\mathbf{x}_k||}$$

Converge rapidissimamente all'autovettore dominante.

Il Metodo delle Potenze è uno degli algoritmi più eleganti dell'algebra lineare numerica. Risponde a una domanda specifica: Come trovo l'autovalore dominante (quello col valore assoluto più grande) senza calcolarli tutti? L'algoritmo si basa su un'intuizione geometrica bellissima. Immagina di avere una matrice $\mathbf{A}$. Un autovettore è un vettore speciale che, se moltiplicato per $\mathbf{A}$, non cambia direzione, ma cambia solo lunghezza (di un fattore pari all'autovalore $\lambda$). Se tu prendi un vettore totalmente casuale $\mathbf{v}_0$ e lo moltiplichi per $\mathbf{A}$, poi ancora per $\mathbf{A}$, poi ancora per $\mathbf{A}$ (da qui "metodo delle potenze", perché calcoli $\mathbf{A}^k\mathbf{v}_0$), cosa succede?

1. Il vettore casuale $\mathbf{v}_0$ può essere visto come un mix (combinazione lineare) di tutti i veri autovettori della matrice.

2. Ogni volta che moltiplichi per $\mathbf{A}$, l'autovettore associato all'autovalore più grande ($\lambda_1$) viene "stirato" più degli altri.

3. Iterazione dopo iterazione, la componente del vettore casuale che punta nella direzione dell'autovalore dominante diventerà sempre più immensa, schiacciando tutte le altre componenti (che sono legate ad autovalori più piccoli).

4. Alla fine, il vettore "punta" esattamente nella direzione dell'autovettore dominante.

Perché normalizziamo ad ogni iterazione? Nel codice abbiamo inserito una divisione per la norma ad ogni ciclo: v->data[i] = w->data[i] / norm;. Se non lo facessimo, continuando a moltiplicare per una matrice, i numeri nel vettore diventerebbero enormi (se $\lambda > 1$) causando un Overflow in MS-DOS, oppure piccolissimi (se $\lambda < 1$) finendo in Underflow e azzerandosi. Normalizzare mantiene il vettore di lunghezza 1 a ogni passo, permettendoci di estrarre l'autovalore calcolando il cosiddetto Quoziente di Rayleigh ($\lambda = \mathbf{v}^T\mathbf{A}\mathbf{v}$). I due limiti del Power Method:

- Trova solo il più grande: Ti dà un solo autovalore (il tuo 16.366). Per trovare il secondo, dovresti usare una tecnica chiamata "Deflation" (sottrarre l'influenza del primo autovalore dalla matrice e ricominciare).

- Velocità di convergenza: Il metodo converge velocemente se c'è un forte "distacco" tra il primo autovalore e il secondo (cioè se $|\lambda_1| \gg |\lambda_2|$). Se i due autovalori più grandi sono quasi identici in valore assoluto (es. 10.0 e 9.9), la convergenza sarà lentissima e il limite di 1000 iterazioni potrebbe non bastare.

L'ottimizzazione per MS-DOS: Invece di usare le funzioni `mat_mul` e `mat_transpose` dentro al ciclo iterativo (che causerebbero continue e lente chiamate a `malloc` e `free`), di seguito e' riportato il calcolo matriciale "inline". Zero allocazioni nel ciclo = maggiore velocità e minore frammentazione della RAM.

```
TYPE eigen_power_method(const mat *A, mat *eigenvec_out, TYPE
    tolerance, int max_iter)
{
        /* * Trova l'autovalore dominante e il suo autovettore.
         * A: Matrice quadrata
```

```
 * eigenvec_out: (Opzionale) Vettore colonna pre-allocato nx1 dove salvare l'autovettore
 * tolerance: Tolleranza per la convergenza (es. 1e-6)
 * max_iter: Limite di sicurezza (es. 1000)
 */
int n = A->n_rows;
mat *v = mat_new(n, 1);
mat *w = mat_new(n, 1);
int i, r, c, iter;
TYPE lambda = 0.0, lambda_old = 0.0, norm;

if(A->n_rows != A->n_cols) return 0.0; // Solo matrici quadrate

// 1. Inizializziamo il vettore v con tutti 1.0 e lo normalizziamo
for(i=0; i<n; i++) v->data[i] = 1.0;
norm = mat_l2norm(v);
for(i=0; i<n; i++) v->data[i] /= norm;

for(iter = 0; iter < max_iter; iter++) {

        // 2. w = A * v (Fatto "inline" per evitare malloc/free lente in DOS)
        for(r = 0; r < n; r++) {
                w->data[r] = 0.0;
                for(c = 0; c < n; c++) {
                        w->data[r] += A->data[r * A->n_cols + c] * v->data[c];
                }
        }

        // 3. Calcolo del Quoziente di Rayleigh (lambda = v^T * w)
        lambda = 0.0;
        for(i = 0; i < n; i++) {
                lambda += v->data[i] * w->data[i];
        }

        // 4. Controllo di convergenza
        if (fabs(lambda - lambda_old) < tolerance) break;
        lambda_old = lambda;

        // 5. Normalizziamo w per ottenere il nuovo v
        norm = mat_l2norm(w);
        if(norm < SLAP_MIN_COEF) break; // Evita divisioni per zero
        for(i = 0; i < n; i++) {
                v->data[i] = w->data[i] / norm;
        }
```

```
        }

        // Se l'utente ha passato un vettore output, salviamo l'autovettore
        if(eigenvec_out && eigenvec_out->n_rows == n && eigenvec_out->n_cols == 1) {
            for(i = 0; i < n; i++) eigenvec_out->data[i] = v->data[i];
        }

        mat_free(v);
        mat_free(w);
        return lambda;
}
```

Questa funzione puo′ essere usata come:

```
TYPE max_eig = eigen_power_method(A, autovettore, 1e-6, 1000);
```

# A Random numbers

Algoritmi di generazione dei numeri casuali

## A.1 Sampling from Gaussian distribution

metodi di sampling descritti sul mio quaderno dei ponti integrali, con qualche indicazione in piu'

# B Stress test

## B.1 Matrici di Hilbert

Le matrici di Hilbert per N molto grande sono particolarmente instabili. Sono definite come

$$H_{i,j} = \frac{1}{i+j-1} \tag{42}$$

Per la proprieta' transitiva tra $i$ e $j$ al denominatore, si vede chiaramente che e' simmetrica ($H_{i,j} = H_{j,i}$). Ad esempio la matrice di Hilbert di ordine 3 e'

$$\mathbf{H}_3 = \begin{bmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{bmatrix}$$

Anche per ordini piccoli ($N = 5$ o $N = 6$) calcolare l'inversa puo' portare a errori di arrotondamento.

## B.2 Matrice quasi instabile

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1.0001 & 1 \\ 1 & 1 & 1.0001 \end{bmatrix}$$

Questa matrice è un altro esempio fantastico, molto diverso da quello di Hilbert, ma altrettanto insidioso. Possiamo scriverla introducendo un piccolo "disturbo" $\epsilon = 0.0001$:

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1+\epsilon & 1 \\ 1 & 1 & 1+\epsilon \end{bmatrix}$$

Se si calcola l'inversa mantenendo $\epsilon$ come variabile, si ottiene una forma analitica molto chiara:

$$A^{-1} = \begin{bmatrix} 1 + \frac{2}{\epsilon} & -\frac{1}{\epsilon} & -\frac{1}{\epsilon} \\ -\frac{1}{\epsilon} & \frac{1}{\epsilon} & 0 \\ -\frac{1}{\epsilon} & 0 & \frac{1}{\epsilon} \end{bmatrix}$$

Sostituendo $\epsilon = 0.0001$ (il cui reciproco $\frac{1}{\epsilon} = 10000$), otteniamo esattamente la matrice indicata sopra. Perché questa matrice è il "boss finale" per la tua libreria SLAP? Se provi a dare questa matrice in pasto al tuo solutore, testerai una cosa cruciale: la tolleranza allo zero.

- **Quasi-Singolarità**: La matrice di partenza è "quasi" una matrice di tutti 1 (che avrebbe determinante pari a 0 e non sarebbe invertibile). Il suo determinante reale è piccolissimo: $\det(A) = \epsilon^2 = 10^{-8}$.

- **Il ruolo di `SLAP_MIN_COEF`**: Nel tuo file `SLAP_merged.h` hai definito `#define SLAP_MIN_COEF (LDBL_EPSILON * 100.0L)`. Dal momento che il determinante scende a $10^{-8}$, algoritmi mal scritti o con tolleranze troppo alte potrebbero credere che la matrice sia singolare e bloccare l'esecuzione ritornando `NULL`.

- **Il vantaggio del tuo long double in DOS**: Fortunatamente, i vecchi compilatori Borland C++ su MS-DOS usano la FPU x87, che mappa il long double a 80-bit in hardware. Questa precisione estesa ti salva la vita qui, garantendo che $10^{-8}$ venga gestito senza collassare nello zero macchina.